



UNIVERSIDADE FEDERAL DE SANTA CATARINA  
CENTRO TECNOLÓGICO DE JOINVILLE  
DEPARTAMENTO DE ENGENHARIAS DA MOBILIDADE

**Trabalho Final em Dupla – Snake Game**

Autores: Esther Duarte dos Reis  
Lucas Turcatto de Carvalho

Orientadora: Arthur Miguel Gabardo

Joinville  
2025

# Sumário

|                                                |           |
|------------------------------------------------|-----------|
| <b>Resumo</b>                                  | <b>3</b>  |
| <b>1 Introdução</b>                            | <b>3</b>  |
| <b>2 Metodologia</b>                           | <b>3</b>  |
| <b>3 Arquitetura do Sistema</b>                | <b>4</b>  |
| 3.1 Diagrama de Arquitetura MVC . . . . .      | 4         |
| 3.2 Hierarquia de Classes . . . . .            | 5         |
| 3.3 Fluxo do Game Loop . . . . .               | 6         |
| <b>4 Demonstração do Jogo</b>                  | <b>8</b>  |
| 4.1 Versão Gráfica (Allegro 5) . . . . .       | 8         |
| 4.2 Versão Texto (Terminal) . . . . .          | 10        |
| 4.3 Controles do Jogo . . . . .                | 11        |
| <b>5 Conceitos de POO Aplicados</b>            | <b>12</b> |
| 5.1 Abstração e Encapsulamento . . . . .       | 12        |
| 5.2 Herança e Polimorfismo . . . . .           | 12        |
| 5.3 Programação Genérica . . . . .             | 12        |
| 5.4 Boas Práticas Aplicadas . . . . .          | 13        |
| 5.5 Tabela de Rastreabilidade . . . . .        | 13        |
| <b>6 Programação Genérica e STL</b>            | <b>13</b> |
| 6.1 Templates no Projeto . . . . .             | 13        |
| 6.2 STL - Containers e Algoritmos . . . . .    | 14        |
| 6.3 Vantagens da Abordagem . . . . .           | 14        |
| <b>7 Detalhes de Implementação</b>             | <b>15</b> |
| 7.1 Três Versões do Jogo . . . . .             | 15        |
| 7.2 Mecânicas do Jogo . . . . .                | 15        |
| 7.3 Sistema de Build e Empacotamento . . . . . | 16        |
| <b>8 Resultados Obtidos</b>                    | <b>16</b> |
| 8.1 Desempenho . . . . .                       | 16        |
| 8.2 Funcionalidades Implementadas . . . . .    | 16        |
| 8.3 Portabilidade . . . . .                    | 16        |
| <b>9 Conclusão e Aprendizados</b>              | <b>17</b> |
| 9.1 Objetivos Alcançados . . . . .             | 17        |

|          |                                   |           |
|----------|-----------------------------------|-----------|
| 9.2      | Aprendizados Principais . . . . . | 17        |
| <b>A</b> | <b>Estrutura do Projeto</b>       | <b>17</b> |
| <b>B</b> | <b>Como Executar o Jogo</b>       | <b>18</b> |

# Resumo

Este trabalho apresenta o desenvolvimento completo do jogo clássico Snake usando C++. Criamos três versões do jogo: uma com gráficos avançados usando Allegro 5, outra versão gráfica sem áudio para ambientes restritos, e uma versão em modo texto para terminal. O projeto demonstra conceitos importantes de Programação Orientada a Objetos como herança, polimorfismo e encapsulamento, além de boas práticas de desenvolvimento como uso de containers Docker para garantir que o jogo funcione em qualquer computador.

**Palavras-chave:** C++; Allegro 5; MVC; CMake; Docker; POO; Desenvolvimento de Jogos.

## 1 Introdução

O **Snake Game** é um dos jogos mais icônicos da história dos videogames, conhecido por sua simplicidade e diversão. Neste projeto, recriamos este clássico para demonstrar como conceitos de programação avançada podem ser aplicados no desenvolvimento de jogos.

### Por que escolhemos o Snake?

- **Simplicidade conceitual:** Fácil de entender, mas desafiador para implementar bem
- **Oportunidade educativa:** Permite demonstrar diversos conceitos de POO
- **Resultado tangível:** Podemos ver o jogo funcionando e compartilhar com outras pessoas

### Objetivos do projeto:

- Aprender e aplicar conceitos de Programação Orientada a Objetos
- Desenvolver habilidades com bibliotecas gráficas (Allegro 5)
- Implementar um sistema de build profissional com CMake
- Garantir portabilidade usando containers Docker
- Criar documentação completa e profissional

## 2 Metodologia

Desenvolvemos o projeto usando uma abordagem iterativa, onde cada funcionalidade foi implementada, testada e integrada gradualmente.

### Ferramentas que usamos:

- **Linguagem:** C++17

- **Gráficos e Áudio:** Allegro 5
- **Build System:** CMake (sistema de compilação)
- **Controle de Versão:** Git (para gerenciar mudanças no código)
- **Containerização:** Docker (para "empacotar" e distribuir o jogo)

## 3 Arquitetura do Sistema

Organizamos nosso código usando o padrão **MVC (Model-View-Controller)**, que separa as responsabilidades em três partes:

- **Model (Modelo):** Gerencia os dados do jogo (Snake, Apple, Field)
- **View (Visualização):** Mostra o jogo na tela (ConsoleHUD, Renderização)
- **Controller (Controlador):** Controla a lógica do jogo (Game)

### 3.1 Diagrama de Arquitetura MVC

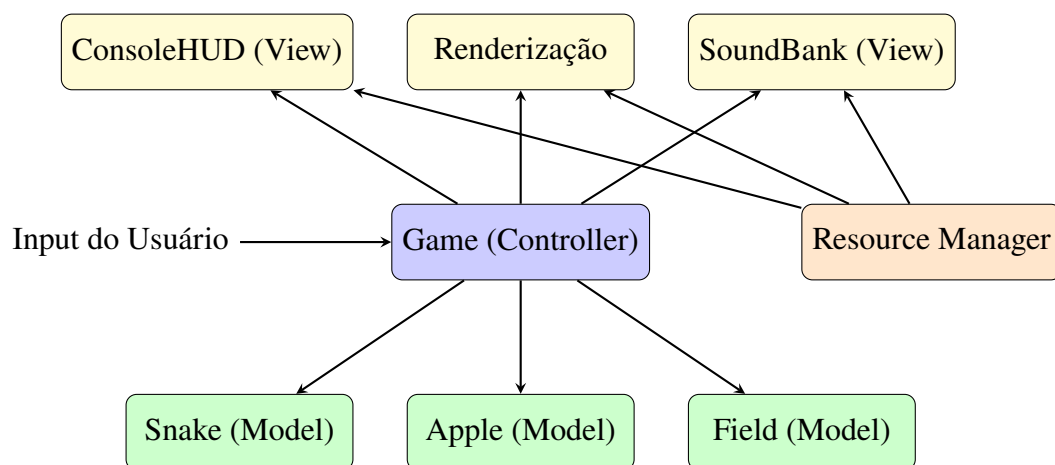


Figura 1: Arquitetura MVC - O Controller (Game) coordena tudo, o Model gerencia os dados, e o View mostra na tela

#### Explicação do diagrama:

- O **Game** é o cérebro do jogo - controla tudo
- **Snake, Apple, Field** são os dados do jogo (onde estão os objetos)
- **ConsoleHUD, Renderização, SoundBank** mostram e tocam o jogo
- O **Resource Manager** carrega imagens, sons e fontes de forma eficiente
- O **Usuário** interage através do teclado

### 3.2 Hierarquia de Classes

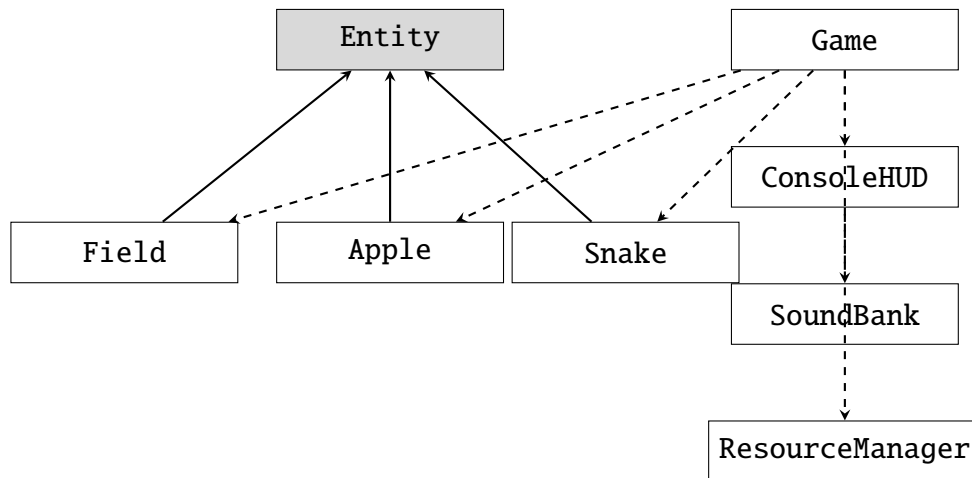


Figura 2: Hierarquia de classes - Entity é a classe pai, e Game coordena todas as outras

#### Explicação das relações:

- **Herança** (setas contínuas): Field, Apple e Snake são tipos de Entity
- **Composição** (setas tracejadas): Game contém e usa todas as outras classes
- Entity é uma classe abstrata que define o que todas as entidades do jogo devem fazer
- Cada classe tem uma responsabilidade específica, seguindo o princípio de responsabilidade única

### 3.3 Fluxo do Game Loop

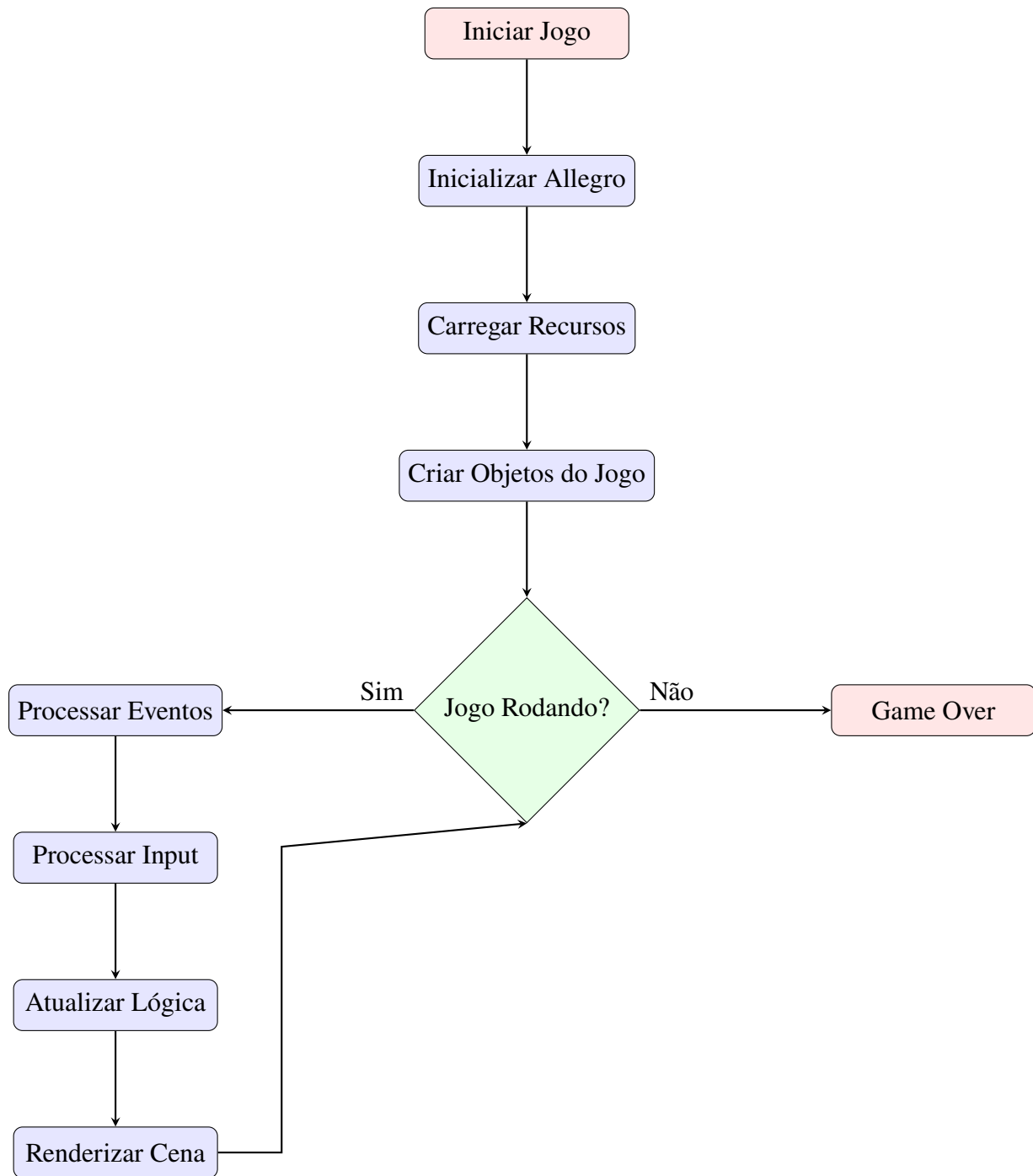


Figura 3: Fluxo principal do jogo - o loop que se repete 60 vezes por segundo atualizando tudo

#### Explicação detalhada do fluxo:

O game loop é o coração do jogo - ele roda continuamente (60 vezes por segundo) e é responsável por manter o jogo funcionando.

#### 1. Inicialização do Jogo:

- **Inicializar Allegro:** Prepara a biblioteca gráfica para funcionar
- **Carregar Recursos:** Carrega imagens, sons e fontes do jogo
- **Criar Objetos:** Cria a cobra, a maçã, o campo e a interface

#### 2. Loop Principal (60 FPS):

- **Processar Eventos:** Verifica se aconteceu algo (fechar janela, etc.)
- **Processar Input:** Lê as teclas pressionadas pelo jogador
- **Atualizar Lógica:** Move a cobra, verifica colisões, atualiza score
- **Renderizar Cena:** Desenha tudo na tela (campo, cobra, maçã, HUD)

#### 3. Verificação Contínua:

- A cada ciclo, verifica se o jogo ainda deve continuar rodando
- Se o jogador fechou a janela ou perdeu o jogo, vai para "Game Over"
- Caso contrário, repete o loop infinitamente

#### Por que 60 FPS?

- **Suavidade:** 60 atualizações por segundo garantem movimento fluido
- **Responsividade:** O jogo responde rapidamente aos comandos
- **Padrão da indústria:** A maioria dos jogos modernos usa 60 FPS

#### O que acontece em cada etapa:

##### Processar Input:

- Detecta se as teclas ←, →, P, C, M, N foram pressionadas
- Atualiza a direção da cobra conforme as setas
- Gerencia pausa, mudança de cor e configurações de áudio

##### Atualizar Lógica:

- **Movimento:** Calcula nova posição da cobra baseada na direção
- **Colisões:** Verifica se a cobra comeu maçã ou bateu em si mesma
- **Crescimento:** Aumenta o tamanho da cobra quando come maçã
- **Wrapping:** Faz a cobra aparecer do outro lado ao sair da tela

##### Renderizar Cena:



- **Limpa a tela:** Prepara para desenhar o novo frame
- **Desenha campo:** Fundo verde com bordas
- **Desenha elementos:** Cobra, maçã, HUD com score
- **Atualiza display:** Mostra tudo na tela do jogador

## 4 Demonstração do Jogo

### 4.1 Versão Gráfica (Allegro 5)

A versão gráfica oferece uma experiência completa com gráficos coloridos, animações suaves e efeitos sonoros.

#### **Características:**

- Gráficos em alta resolução com sprites detalhados
- Sistema de cores customizável para a cobra
- Heads-up Display (HUD) profissional com score e recorde
- Efeitos sonoros para comer maçã e game over
- Música de fundo atmosférica
- Transições suaves entre menus

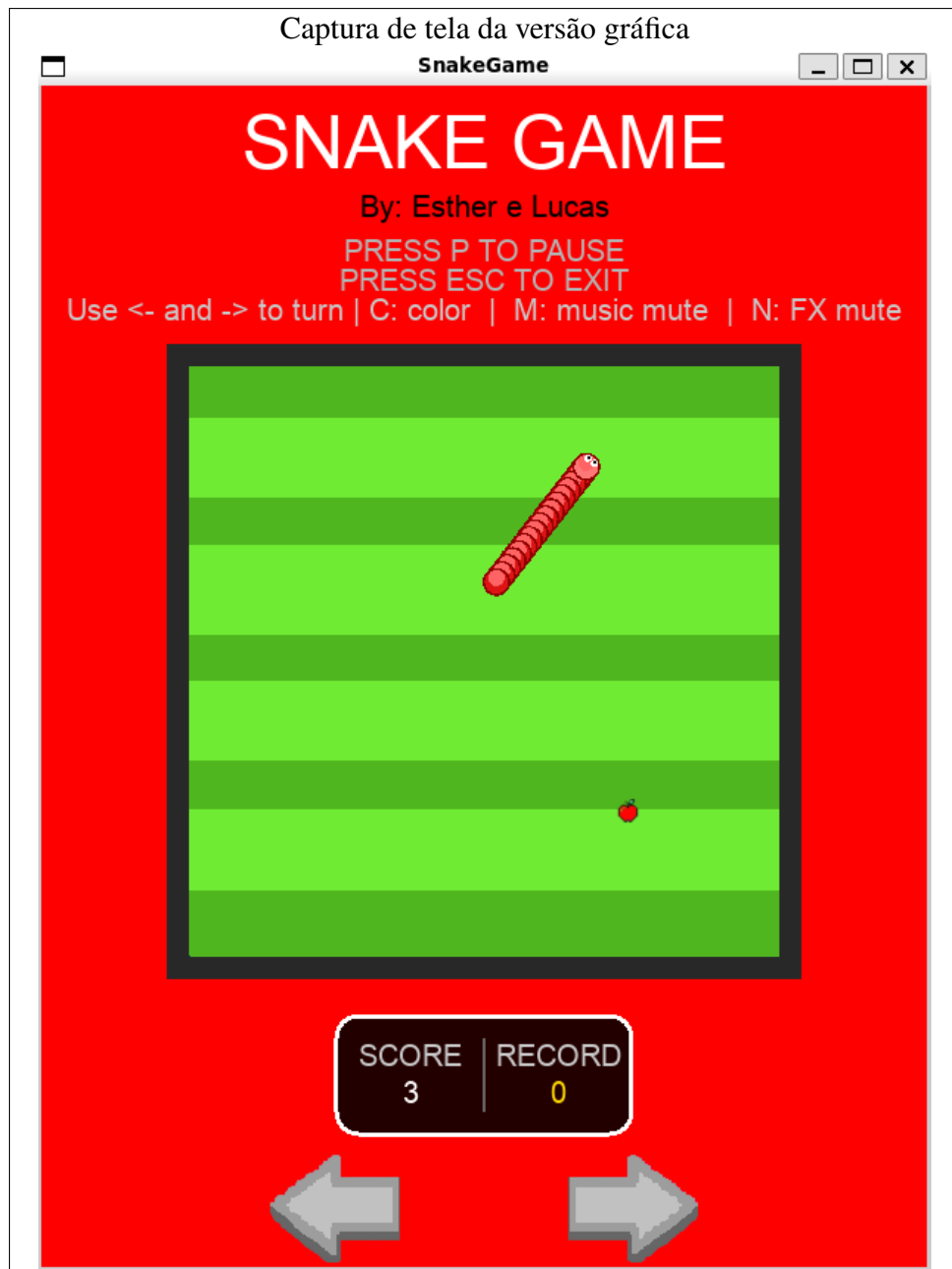


Figura 4: Versão gráfica do jogo com interface colorida e HUD

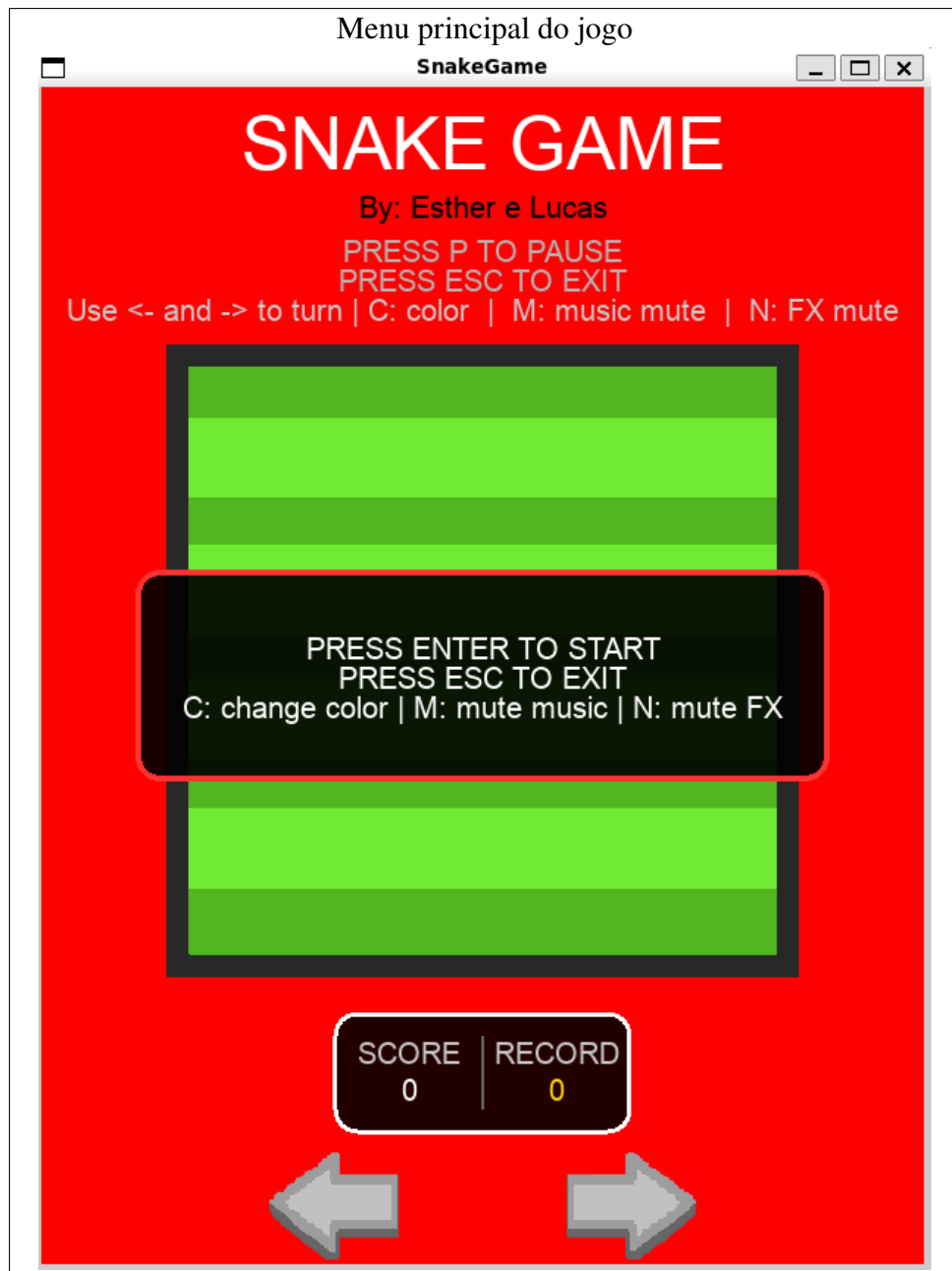


Figura 5: Menu principal com opções de configuração

### 4.2 Versão Texto (Terminal)

A versão texto é perfeita para jogar no terminal ou em ambientes sem interface gráfica.

#### Características:

- Funciona em qualquer terminal
- Controles simples com teclado
- Performance excelente
- Ideal para servidores ou SSH

- Compatível com Windows, Linux e Mac



Figura 6: Versão texto do jogo rodando no terminal

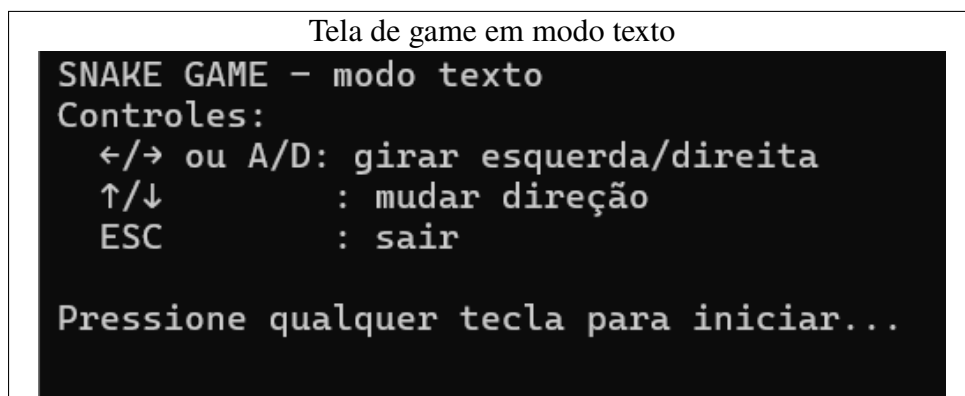


Figura 7: Tela do jogo em execução versão texto

### 4.3 Controles do Jogo

#### Versão Gráfica:

- **Setas Esquerda/Direita:** Virar a cobra
- **P:** Pausar o jogo
- **C:** Mudar cor da cobra
- **M:** Ligar/desligar música
- **N:** Ligar/desligar efeitos sonoros

- **ESC:** Sair do jogo

### **Versão Texto:**

- **A/D ou Setas:** Virar a cobra
- **W/S:** Mover para cima/baixo (opcional)
- **ESC:** Sair do jogo

## 5 Conceitos de POO Aplicados

Neste projeto, aplicamos vários conceitos importantes de Programação Orientada a Objetos:

### 5.1 Abstração e Encapsulamento

**Abstração** significa esconder detalhes complexos e mostrar apenas o necessário. **Encapsulamento** significa proteger os dados internos das classes.

#### **Exemplos no nosso código:**

- A classe `Snake` tem método `grow()` - você não precisa saber **COMO** a cobra cresce, apenas **CHAMA** o método
- A classe `Game` tem `run()` - inicia o jogo sem você precisar entender todo o loop interno
- Todas as variáveis importantes são `private` - protegidas contra acesso incorreto

### 5.2 Herança e Polimorfismo

**Herança** permite que classes compartilhem código comum. **Polimorfismo** permite tratar objetos diferentes de forma uniforme.

#### **Como usamos:**

- `Field`, `Apple` e `Snake` herdam de `Entity`
- Todas implementam `draw()` e `update()` de forma diferente
- O `Game` pode tratar todas como `Entity` sem saber qual é qual

### 5.3 Programação Genérica

Usamos **templates** para criar código reutilizável.

**Exemplo:** `ResourceManager` funciona com qualquer tipo de recurso (imagens, fontes, sons) sem precisar reescrever código.

## 5.4 Boas Práticas Aplicadas

- **Princípio de Liskov:** Nossas subclasses não "quebram" o comportamento esperado
- **Regra dos Três/Cinco:** Classes com recursos gerenciam corretamente memória
- **Responsabilidade Única:** Cada classe tem uma única responsabilidade

## 5.5 Tabela de Rastreabilidade

| Conceito de POO        | Onde implementamos no projeto                                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Abstração (6+ classes) | Game, Snake, Apple, Field, ConsoleHUD, SoundBank - cada uma representa um conceito do jogo                                    |
| Encapsulamento         | Variáveis <code>private</code> e métodos <code>public</code> - como <code>Snake::grow()</code> esconde a complexidade interna |
| Herança                | <code>Entity</code> → <code>Field</code> / <code>Apple</code> / <code>Snake</code> - compartilham código comum                |
| Polimorfismo estático  | <code>Vector2D</code> com operadores <code>+</code> , <code>-</code> , <code>*</code> - mesma operação, tipos diferentes      |
| Polimorfismo dinâmico  | <code>Game</code> trata todas as <code>Entity</code> igualmente, mesmo sendo tipos diferentes                                 |
| Programação genérica   | <code>ResourceManager</code> funciona com qualquer tipo de recurso                                                            |
| Princípio de Liskov    | Subclasses de <code>Entity</code> não quebram o contrato das funções                                                          |
| Regra dos 3/5          | <code>SoundBank</code> gerencia memória de audio corretamente                                                                 |

Tabela 1: Como aplicamos cada conceito de POO no nosso projeto

## 6 Programação Genérica e STL

Usamos extensivamente **Templates** e a **Standard Template Library (STL)** para criar código reutilizável e eficiente.

### 6.1 Templates no Projeto

**Templates** são como "receitas" que funcionam com qualquer tipo. Criamos um `ResourceManager` genérico:

```
1 template<typename Key, typename Resource, typename Loader, typename  
   ↳ Unloader>  
2 class ResourceManager {  
3     std::unordered_map<Key, Resource> cache_;  
4 public:
```

```
5 Resource get(const Key& key) {  
6     // Busca no cache ou carrega novo recurso  
7     return cache_.emplace(key, loader(key)).first->second;  
8 }  
9 };
```

**Uso no projeto:**

- **Imagens:** ResourceManager<string, ALLEGRO\_BITMAP\*, BitmapLoader>
- **Fontes:** ResourceManager<string, ALLEGRO\_FONT\*, FontLoader>
- **Mesmo código** para diferentes tipos de recursos

## 6.2 STL - Containers e Algoritmos

**Containers utilizados:**

- **std::vector:** Segmentos da cobra (acesso rápido  $O(1)$ )
- **std::deque:** Cobra versão texto (inserção rápida nas pontas)
- **std::unordered\_map:** Cache de recursos (busca  $O(1)$  por nome)
- **std::unique\_ptr:** Gerenciamento automático de memória

**Algoritmos STL:**

- **std::max:** Atualizar recorde do jogo
- **std::find:** Verificar colisões
- **std::cos/std::sin:** Movimento da cobra

## 6.3 Vantagens da Abordagem

- **Reutilização:** Um código serve para múltiplos tipos
- **Segurança:** Compilador verifica tipos em tempo de compilação
- **Performance:** Templates resolvidos em compilação (zero overhead)
- **Produtividade:** STL fornece estruturas otimizadas e testadas

**Exemplo prático:**

```
1 // SEM template - c digo duplicado
2 class BitmapManager { ... }; // S bitmaps
3 class FontManager { ... }; // S fonts
4
5 // COM template - c digo reutilizavel
6 template<typename T> class ResourceManager { ... }; // Todos os
    ↪ tipos!
```

A programação genérica nos permitiu escrever menos código com mais segurança e performance, seguindo as melhores práticas do C++ moderno.

## 7 Detalhes de Implementação

### 7.1 Três Versões do Jogo

Desenvolvemos três versões para atender diferentes necessidades:

- **SnakeGame**: Versão completa com gráficos e áudio
- **SnakeGameNoAudio**: Versão gráfica sem áudio (para Docker)
- **SnakeText**: Versão terminal (para ambientes restritos)

### 7.2 Mecânicas do Jogo

#### Movimento da Cobra:

- Rotação suave - a cobra vira gradualmente
- Movimento contínuo - não anda em "casas" como no jogo original
- Wrapping - quando sai de um lado, aparece do outro

#### Sistema de Pontuação:

- +1 ponto por maçã comida
- Recorde salvo automaticamente
- Crescimento progressivo da cobra

#### Deteção de Colisão:

- Com a própria cauda - game over
- Com as maçãs - crescimento e pontos
- Sistema preciso usando matemática vetorial



### 7.3 Sistema de Build e Empacotamento

**CMake:** Sistema profissional para compilar o código em diferentes plataformas.

**Docker:** "Empacotamos" o jogo em containers para garantir que funcione em qualquer computador.

**Como executar:**

# Método fácil (recomendado):

```
./run.sh
```

# Métodos manuais:

```
docker compose up snake-game      # Versão gráfica
```

```
docker compose up snake-text      # Versão texto
```

## 8 Resultados Obtidos

### 8.1 Desempenho

O jogo roda suavemente a **60 FPS** (quadros por segundo) em todos os sistemas testados.

### 8.2 Funcionalidades Implementadas

- Três versões funcionais do jogo
- Controles responsivos e precisos
- Sistema de áudio completo
- Interface gráfica profissional
- Sistema de menus e game states
- Detecção de colisão precisa
- Sistema de score e recordes

### 8.3 Portabilidade

O jogo funciona perfeitamente em:

- Windows 10/11 (com Docker)
- Linux Ubuntu/Debian (nativo e Docker)
- MacOS (com Docker)
- Qualquer sistema com Docker instalado

## 9 Conclusão e Aprendizados

### 9.1 Objetivos Alcançados

Desenvolvemos com sucesso um jogo Snake completo, demonstrando domínio de conceitos de POO na prática, habilidade com bibliotecas gráficas (Allegro 5), uso de sistemas de build profissionais (CMake) e containerização com Docker.

### 9.2 Aprendizados Principais

- **Arquitetura de software:** MVC torna o código mais organizado
- **POO na prática:** Herança, polimorfismo e encapsulamento são úteis
- **Gerenciamento de recursos:** Memória, arquivos, áudio
- **Portabilidade:** Docker resolve muitos problemas de compatibilidade

## A Estrutura do Projeto

```
SnakeGame/  
  Dockerfile           # Configuração do Docker  
  docker-compose.yml   # Orquestração de containers  
  run.sh               # Script de execução fácil  
  README_DOCKER.md     # Instruções de uso  
  CMakeLists.txt       # Configuração de compilação  
  src/  
    main.cpp           # Versão gráfica com áudio  
    main_no_audio.cpp  # Versão gráfica sem áudio  
    main_text_linux.cpp # Versão texto  
    Game.cpp           # Lógica principal do jogo  
    Snake.cpp           # Implementação da cobra  
    Apple.cpp          # Implementação da maçã  
    Field.cpp          # Campo de jogo  
    ConsoleHUD.cpp     # Interface do usuário  
    Sound.cpp          # Sistema de áudio  
    private/  
  include/             # Cabeçalhos das classes  
  assets/  
    Images/            # Sprites e texturas
```

|         |                            |
|---------|----------------------------|
| Sounds/ | # Efeitos sonoros e música |
| Fonts/  | # Fontes de texto          |

## B Como Executar o Jogo

**Pré-requisitos:** Apenas Docker instalado no computador.

**Execução fácil (recomendado):**

```
./run.sh
```

**Execução manual:**

- **Versão Gráfica:** `docker compose up snake-game`
- **Versão Sem Áudio:** `docker compose up snake-game-no-audio`
- **Versão Texto:** `docker compose up snake-text`

**Compilação manual (para desenvolvedores):**

```
cd build
cmake ..
make -j4
./bin/SnakeGame
```